```
/**
 * compliance-and-channels-fixes.patch.ts
 *
 * Fix #7 — Compliance Blocking With No Detail on What's Wrong
 * Fix #8 — `channels` Table Full Scan in Live Detection
 *
 * ═══════════════════════════════════════════════════════════════════════════
 * FIX #7 — COMPLIANCE DETAIL LOGGING + AUTO-FIX ROUTING
 * ═══════════════════════════════════════════════════════════════════════════
 *
 * PROBLEM: The current log only shows:
 *   [autopilot] Content blocked: postId 28347, violations: ["missing_disclosure"]
 *
 * No detail on WHICH video, WHAT disclosure is missing, or HOW to fix it.
 * 21 videos are pending monetization review with no visibility into why.
 *
 * SOLUTION: Enhanced compliance logging that surfaces actionable details
 * and routes each violation type to an auto-fix handler.
 */

import { storage }     from '../storage.js';
import { createLogger } from '../lib/logger.js';

const log = createLogger('compliance-engine');

// ─── Violation types and their auto-fix strategies ──────────────────────────

type ViolationType =
  | 'missing_disclosure'
  | 'missing_description'
  | 'invalid_keywords'
  | 'age_restriction_required'
  | 'monetization_not_enabled'
  | 'copyright_claim_active'
  | 'community_guidelines_strike'
  | string;

interface ComplianceFix {
  type:        ViolationType;
  description: string;
  autoFixable: boolean;
  fixAction:   string;
}

const VIOLATION_REGISTRY: Record<string, ComplianceFix> = {
  missing_disclosure: {
    type:        'missing_disclosure',
    description: 'Video description missing required paid promotion disclosure',
```

```
    autoFixable: true,
    fixAction:   'Append "#Ad" or "Paid promotion" to description via videos.update',
  },
  missing_description: {
    type:        'missing_description',
    description: 'Video has no description — required for SEO and compliance',
    autoFixable: true,
    fixAction:   'Generate SEO description via AI and set via videos.update',
  },
  invalid_keywords: {
    type:        'invalid_keywords',
    description: 'Video tags contain invalid characters or exceed length limits',
    autoFixable: true,
    fixAction:   'Run sanitizeYouTubeTags() and update via videos.update',
  },
  monetization_not_enabled: {
    type:        'monetization_not_enabled',
    description: 'Video not enabled for monetization — channel may not meet YPP threshold',
    autoFixable: false,
    fixAction:   'Manual: enable monetization in YouTube Studio for this video',
  },
  copyright_claim_active: {
    type:        'copyright_claim_active',
    description: 'Active Content ID claim on this video — revenue being redirected',
    autoFixable: false,
    fixAction:   'Manual: review claim in YouTube Studio → Content → Copyright claims',
  },
};

// ─── Enhanced compliance check with full detail logging ──────────────────────

interface ComplianceCheckResult {
  passed:     boolean;
  violations: Array<{
    type:        string;
    description: string;
    autoFixable: boolean;
    fixAction:   string;
  }>;
  autoFixableCount: number;
  manualReviewCount: number;
}

export function evaluateComplianceViolations(
  postId:     number,
  videoId:    number | string,
  youtubeId:  string,
  violations: string[],
): ComplianceCheckResult {
  const enriched = violations.map(v => {
```

```
    const known = VIOLATION_REGISTRY[v];
    return known ?? {
      type:        v,
      description: `Unknown violation type: ${v}`,
      autoFixable: false,
      fixAction:   'Manual review required — check YouTube Studio',
    };
  });

  const autoFixable    = enriched.filter(v => v.autoFixable);
  const manualRequired = enriched.filter(v => !v.autoFixable);
  const passed         = violations.length === 0;

  if (!passed) {
    log.warn(`[Compliance] BLOCKED postId=${postId} videoId=${videoId} youtubeId=${youtubeId}`,
{
      violationCount:    violations.length,
      autoFixableCount:  autoFixable.length,
      manualReviewCount: manualRequired.length,
      violations: enriched.map(v => ({
        type:        v.type,
        description: v.description,
        autoFixable: v.autoFixable,
        fixAction:   v.fixAction,
      })),
    });

    if (autoFixable.length > 0) {
      log.info(
        `[Compliance] ${autoFixable.length} violation(s) on postId=${postId} are AUTO-FIXABLE. `
+
        `Actions: ${autoFixable.map(v => v.fixAction).join(' | ')}`
      );
    }

    if (manualRequired.length > 0) {
      log.warn(
        `[Compliance] ${manualRequired.length} violation(s) on postId=${postId} require MANUAL
REVIEW: ` +
        `${manualRequired.map(v => v.type).join(', ')}`
      );
    }
  }

  return {
    passed,
    violations:       enriched,
    autoFixableCount: autoFixable.length,
    manualReviewCount: manualRequired.length,
  };
```

```
}

// ─── Usage — replace current compliance check in autopilot.ts ──────────────
//
// BEFORE:
//   if (violations.length > 0) {
//     log.warn('Content blocked by compliance check', { postId, platform, violations });
//     return;
//   }
//
// AFTER:
//   const compliance = evaluateComplianceViolations(postId, videoId, youtubeId, violations);
//   if (!compliance.passed) {
//     // Queue auto-fixable violations for the content-grinder to fix
//     if (compliance.autoFixableCount > 0) {
//       await storage.queueComplianceFix(postId, compliance.violations.filter(v =>
v.autoFixable));
//     }
//     return;
//   }


// ═══════════════════════════════════════════════════════════════════════════
// FIX #8 — channels TABLE FULL SCAN IN LIVE DETECTION
// ═══════════════════════════════════════════════════════════════════════════
//
// PROBLEM: The live detection query runs with EMPTY params (no WHERE clause),
// doing a full table scan on every detection cycle. Under connection pressure,
// this is the query that kills the pool:
//
//   [LiveDetection] Failed query:
//     select "id", "user_id", "platform", ... from "channels"
//     params:                                          ← EMPTY! No WHERE clause
//
// This is reading ALL channels (YouTube + TikTok + Rumble + Kick + Twitch)
// when it only needs YouTube channels.
//
// BEFORE (in server/services/live-detection.ts or similar):
//   const channels = await db.select().from(channelsTable);
//   // OR
//   const channels = await storage.getAllChannels();
//
// AFTER — add platform filter:

import { db }            from '../db.js';
import { channels }      from '../../shared/schema.js';
import { eq }            from 'drizzle-orm';

export async function getActiveYouTubeChannels() {
  // Platform-filtered query — only reads YouTube channels
```

```
  // Prevents full table scan that was killing the connection pool
  return db
    .select({
      id:             channels.id,
      userId:         channels.userId,
      platform:       channels.platform,
      channelName:    channels.channelName,
      channelId:      channels.channelId,
      accessToken:    channels.accessToken,
      refreshToken:   channels.refreshToken,
      tokenExpiresAt: channels.tokenExpiresAt,
    })
    .from(channels)
    .where(eq(channels.platform, 'youtube')); // ← THE FIX: was missing this WHERE
}

// Also add an index on the channels table if it doesn't exist:
//   CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_channels_platform
//   ON channels (platform);
//
// This makes the platform filter instantaneous even as the table grows.
//
// Similarly, fix pipeline-tracer which shows the same pattern:
//   [PipelineTracer] Failed query:
//     select "user_id", "id" from "channels"
//     WHERE "channels"."platform" = $1  ← this one HAS the WHERE, good
//     params: youtube
//
// The pipeline-tracer query is correct already — the failure is pure
// connection pool exhaustion, not a missing filter. It will resolve
// once the boot DB ready check and AI saturation fixes are in place.
```